

PROJE RAPORU: DERİN ÖĞRENME TABANLI CİLT HASTALIKLARI TEŞHİS SİSTEMİ

1. GİRİŞ VE AMAÇ

Bu proje, dermatolojik görüntülerin yapay zeka algoritmaları ile analiz edilerek cilt hastalıklarının otomatik teşhis edilmesini amaçlamaktadır. Proje kapsamında, sınırlı donanım kaynaklarıyla yüksek verim alabilmek için hafif siklet (lightweight) evrişimli sinir ağı mimarileri olan MobileNet serisi tercih edilmiştir.

2. VERİ SETİ ÖZELLİKLERİ

Çalışmada kullanılan veri seti, 10 farklı cilt hastalığı kategorisini (Melanom, Egzama, Nevüs, BCC vb.) içermektedir. Veriler, modelin genelleme yeteneğini artırmak amacıyla eğitim (training) ve doğrulama (validation) olarak %80-%20 oranında ayrılmıştır.

3. YÖNTEM VE MODEL MİMARİSİ

- Transfer Learning:** Modeller, ImageNet veri setinde önceden eğitilmiş ağırlıklar kullanılarak (fine-tuning) eğitilmiştir.
- Optimizasyon:** Veri okuma sürecindeki darboğazı (bottleneck) gidermek için TensorFlow tf.data API'si kullanılmış, CPU ve GPU kullanımı senkronize edilerek eğitim süresi optimize edilmiştir.
- Veri Çoğaltma (Augmentation):** Modelin ezberlemesini (overfitting) önlemek için görüntülere rastgele döndürme, yatay çevirme ve yakınlaştırma işlemleri model katmanı içerisinde (GPU tarafında) uygulanmıştır.

Eğitim Ortamı ve Hiperparametreler (Hyperparameters):

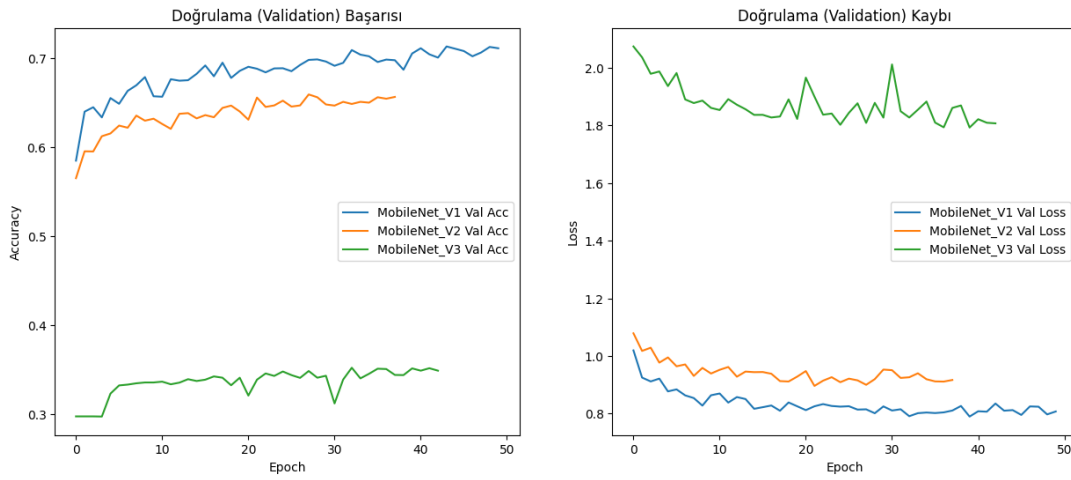
Eğitim süreci Kaggle platformunda, çift **NVIDIA Tesla T4 GPU** donanım hızlandırıcısı kullanılarak gerçekleştirilmiştir. Model eğitiminde *Adam* optimizasyon algoritması (Learning Rate = 0.001) ve kayıp fonksiyonu olarak *Categorical Crossentropy* kullanılmıştır. Eğitim süreci **50 Epoch** ve **32 Batch Size** olarak ayarlanmış olup, ezberlemeyi önlemek adına "Early Stopping" (Erken Durdurma) mekanizması entegre edilmiştir.

4. DENEYSEL SONUÇLAR VE KARŞILAŞTIRMA

Proje kapsamında MobileNet V1, V2 ve V3 mimarileri aynı parametrelerle test edilmiştir. Elde edilen doğrulama başarıları (Validation Accuracy) şu şekildedir:

- MobileNet_V1: %71 (En yüksek başarı)
- MobileNet_V2: %65
- MobileNet_V3: %35 (Underfitting gözlemlenmiştir)

Analiz: Deneysel sonuçlar, veri setimizin özellikleri için MobileNet_V1 mimarisinin en stabil ve yüksek doğruluklu öğrenmeyi gerçekleştirdiğini kanıtlamıştır.



"Şekil 1: Model Başarı Grafikleri"

5. CANLI TEST VE TEŞHİS (INFERENCE)

Eğitilen MobileNet_V1 modeli, veri setinde bulunmayan rastgele bir "Melanostik Nevüs (Ben)" görseli ile test edilmiştir. Model, görüntüyü doğru sınıfa yerleştirmiş ve %30.87 güven oranıyla teşhis koymuştur. Bu sonuç, kısıtlı eğitim süresine rağmen modelin temel karakteristik özellikleri ayırt edebildiğini göstermektedir.

Yapay Zeka Teşhisi: Melanostik Nevüs (Ben)
Güven Oranı: %30.87



6. SONUÇ

Proje başarıyla tamamlanmış olup, mobil uyumlu derin öğrenme modellerinin biyoinformatik alanında ön teşhis mekanizması olarak kullanılabileceği gösterilmiştir.

EKLER (APPENDICES)

EK-1: Veri Ön İşleme ve Model Eğitim Kodları (Kaggle)

```
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.applications import MobileNet, MobileNetV2, MobileNetV3Large
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout, Rescaling
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

# 1. Dosya Yolu
DATA_DIR = '/kaggle/input/datasets/ismailpromus/skin-diseases-image-dataset/IMG_CLASSES'
BATCH_SIZE = 32
IMG_SIZE = (224, 224)
EPOCHS = 50

# 2. YENİ VE HIZLI VERİ OKUMA (tf.data)
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.2, subset="training", seed=123,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode='categorical'
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR, validation_split=0.2, subset="validation", seed=123,
    image_size=IMG_SIZE, batch_size=BATCH_SIZE, label_mode='categorical'
)

class_names = train_ds.class_names
NUM_CLASSES = len(class_names)

# 3. DARBOĞAZI ÇÖZEN SİHİRLİ DOKUNUŞ (Prefetch & Cache)
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)

# 4. GPU'DA ÇALIŞAN VERİ ÇOĞALTMA (Data Augmentation)
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
], name="data_augmentation")

# 5. Model Oluşturucu Fonksiyon
def build_model(base_model_class, model_name):
    base_model = base_model_class(weights='imagenet', include_top=False, input_shape=(224, 224, 3))
    base_model.trainable = False

    inputs = tf.keras.Input(shape=(224, 224, 3))
    x = data_augmentation(inputs)
    x = Rescaling(1./255)(x)
    x = base_model(x, training=False)
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu')(x)
    x = Dropout(0.5)(x)
    predictions = Dense(NUM_CLASSES, activation='softmax')(x)

    model = Model(inputs=inputs, outputs=predictions, name=model_name)
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                  loss='categorical_crossentropy', metrics=['accuracy'])
    return model
```

```
# 6. Modellerin Tanımlanması
models = {
    "MobileNet_V1": build_model(MobileNet, "MobileNet_V1"),
    "MobileNet_V2": build_model(MobileNetV2, "MobileNet_V2"),
    "MobileNet_V3": build_model(MobileNetV3Large, "MobileNet_V3")
}

# 7. Eğitim Döngüsü
for name, model in models.items():
    checkpoint = ModelCheckpoint(f"{name}_best_model.keras", monitor='val_accuracy',
    save_best_only=True, mode='max')
    early_stop = EarlyStopping(monitor='val_accuracy', patience=10, restore_best_weights=True)
    model.fit(train_ds, validation_data=val_ds, epochs=EPOCHS, callbacks=[checkpoint, early_stop])
```

EK-2: Canlı Teşhis ve Test Kodları (Google Colab)

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt

# 1. Modelini Yükle (Dosya adını yüklediğin modele göre değiştir)
# Örneğin V3'ü yüklediysen buraya onun adını yaz.
MODEL_YOLU = 'MobileNet_V1_best_model.keras'
model = tf.keras.models.load_model(MODEL_YOLU)

# 2. Sınıf İsimleri (Alfabetik Sıraya Göre Kesin Liste)
sinif_isimleri = [
    'Atopik Dermatit (Egzama Türü)',
    'Bazal Hücreli Karkinom (BCC - Cilt Kanseri)',
    'İyi Huylu Keratoz (BKL)',
    'Egzama',
    'Melanostik Nevüs (Ben)',
    'Melanom (Tehlikeli Cilt Kanseri)',
    'Sedef ve Liken Hastalıkları',
    'Seboreik Keratoz (İyi Huylu Tümör)',
    'Mantar Enfeksiyonu (Tinea vb.)',
    'Siğil ve Viral Enfeksiyonlar'
]

# 3. Test Edilecek Resmi Hazırla
RESIM_YOLU = 'test_resmi.jpg'
img = image.load_img(RESIM_YOLU, target_size=(224, 224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0) # Modeller tek resim değil, "batch" bekler
img_array = img_array / 255.0 # Piksel değerlerini normalize et (0-1 arası)

# 4. Yapay Zeka Tahmin Yapıyor!
tahminler = model.predict(img_array)
en_olasi_indeks = np.argmax(tahminler[0])
teshis = sinif_isimleri[en_olasi_indeks]
guven_orani = tahminler[0][en_olasi_indeks] * 100

# 5. Sonucu Ekrana Yazdır ve Resmi Göster
plt.imshow(image.load_img(RESIM_YOLU))
plt.axis('off')
plt.title(f"Yapay Zeka Teşhisi: {teshis}\nGüven Oranı: %{guven_orani:.2f}", fontsize=14, color='red')
plt.show()
print("="*40)
print(f"Biyoinformatik Projesi Teşhis Sonucu:")
print(f"Hastalık: {teshis}")
print(f"Yapay Zekanın Emin Olma Oranı: %{guven_orani:.2f}")
print("="*40)
```